



Institut Africain d'Informatique
Email : contact@iai-siege.com
Tel : (+241) 77 70 55 00
Site web : www.iai-siege.com
BP 2263, IAI, Libreville - GABON



RAPPORT DE TP DE MONITORING

Projet 10 — Déploiement Django avec Nginx + Gunicorn + PostgreSQL

Classe : ING 2

Matière : Sécurité des Réseaux

Superviseur :

M. Brice ONDJIBOU

Réalisé par :

WILSON-BAHUN Adjévi Mawougnon

DANSOU Koffi Junior

MAHANGA MBOUMBA Jamarthe Celly

EYA'A BIYOGHE Norwen Geoffrey

Année académique : 2025 -2026

Tables des figures

Figure 1 - Prérequis communs (toutes les machines).....	11
Figure 2 - Installation docker (machines NOC & APP).....	12
Figure 3 - Vérification Docker	12
Figure 4 - Vérification des services natifs sur nocserv	16
Figure 5 - Vérification des services natifs sur appserv	17
Figure 6 - Vérification des services natifs sur dbserve	17
Figure 7 - Vérification des conteneurs docker sur nocserv	17
Figure 8 - Vérification des endpoints de santé	18
Figure 9 - Vérification des targets Prometheus	18
Figure 10 - Dashboard USE interface dans Grafana	20
Figure 11 - Dashboard RED : Application Django.....	22
Figure 12 - Dashboard PostgreSQL.....	23
Figure 13 - Dashboard SLO Error budget	25

Sommaire

Tables des figures.....	2
1. Présentation du projet	4
2. Architecture globale du NOC.....	7
3. Préparation de l'environnement (Debian + Docker).....	11
4. Description détaillée des services et outils	13
5. Guide d'installation et de configuration des outils + vérification	16
6. L3 — Les 7 dashboards Grafana	19
7. Bonus — Déploiement Ansible (IaC).....	27
Conclusion	28
Table des matières	29

1. Présentation du projet

1.1. Contexte pédagogique

Ce projet constitue le livrable final du module Supervision & Observabilité de 2e année Génie Logiciel à l'IAI Gabon. Il consiste à concevoir, déployer et opérer un NOC (Network Operations Center) complet où Grafana joue le rôle de centre de commandement unique. L'ensemble des sources de données — métriques, logs, traces, événements de sécurité — converge vers une interface Grafana unifiée, éliminant tout outil de visualisation externe.

1.2. Mise en situation

EService Linux déploie une infrastructure e-commerce composée de 3 serveurs Linux, d'une application Django, d'une application Laravel, d'une base de données PostgreSQL, d'un reverse proxy Nginx et de conteneurs Docker. La mission est de construire le NOC complet avec Grafana comme unique interface de supervision pour les équipes SRE, NOC opérationnel et direction.

1.3. Stack technique

Composant	Version	Rôle
Grafana	13.0.2	Hub central — visualisation, alerting, runbooks
Prometheus	3.5.3	Scraping et stockage des métriques
Loki	3.7.2	Agrégation et indexation des logs
Tempo	3.0.2	Collecte et stockage des traces distribuées
AlertManager	0.33.0	Routage et déduplication des alertes
Wazuh	4.14.5	SIEM, détection d'intrusion, FIM
Elasticsearch	8.14.0	Indexation des événements de sécurité
Node Exporter	1.11.0	Métriques système Linux

Composant	Version	Rôle
cAdvisor	0.60.1	Métriques des conteneurs Docker
Django	4.x	Application web principale
PostgreSQL	16+	Base de données relationnelle
Nginx	1.25+	Reverse proxy
Docker + Compose	25+	Orchestration des conteneurs
Debian	13 (Bookworm)	OS hôte des VMs

1.4. Livrables attendus

Livrable	Description	Format	Points
L1	Architecture NOC complète (schéma + justifications)	PDF 15–20 pages	3/20
L2	Fichiers de configuration (Prometheus, Loki, Tempo, Wazuh, AlertManager)	GitHub repo	4/20
L3	8 dashboards Grafana (USE, RED, SOC, SLO, DB, Pipeline, Exécutif, K8s)	JSON + captures	4/20
L4	5 runbooks visuels Grafana	Markdown + captures	3/20
L5	3 postmortems complets format SRE Google	PDF	3/20
L6	Playlist NOC kiosk mode + rapport hebdomadaire	Captures + PDF	3/20
L7	Présentation finale 30 min	Oral + slides Grafana	Bonus

1.5. Environnement de travail

- Machine hôte : Windows 11

- VMs
 - Serveur NOC : Debian 13 (Trixie) – 4 vCPU, 8 Go RAM minimum recommandés
 - Serveur d'application : Debian 13 (Trixie) – 2 vCPU, 2 Go RAM
 - Serveur de BD : Debian 13 (Trixie) – 2 vCPU, 2 Go RAM
- Accès client : navigateur Windows → IP de la VM sur port 3000 (Grafana)
- Tout le projet tourne dans Docker Compose sur la VM Debian

Conseil : réserver au moins 6 Go de RAM à la VM. Wazuh + Elasticsearch consomment à eux seuls environ 3 Go.

2. Architecture globale du NOC

2.1. Principe architectural

L'architecture du NOC repose sur quatre couches distinctes : les sources de données, les collecteurs spécialisés, Grafana comme hub central de visualisation et d'alerting, et enfin les équipes consommatrices. Ce modèle en étoile garantit que toute information de supervision transite par un point unique, réduisant la fragmentation des outils et facilitant la corrélation d'événements.

2.2. Les quatre couches

a. Couche 1 — Sources

- Node Exporter : expose les métriques système Linux (CPU, RAM, disque, réseau) sur /metrics
- cAdvisor : expose les métriques de chaque conteneur Docker
- django-prometheus : expose les métriques applicatives de Django (requêtes, erreurs, durée)
- postgres_exporter : expose les métriques PostgreSQL (connexions, transactions, WAL, locks)
- Nginx (via stub_status ou OpenTelemetry) : métriques de trafic HTTP
- Wazuh Agent : collecte les événements de sécurité, FIM, logs système

b. Couche 2 — Collecteurs

- Prometheus : scrape toutes les sources métriques selon prometheus.yml, stocke en TSDB locale
- Promtail : collecte les fichiers de logs (Django, Nginx, système) et les pousse vers Loki
- Loki : agrège et indexe les logs pour requêtage via LogQL dans Grafana
- OTel Collector : reçoit les traces de Django (instrumentation OpenTelemetry) et les transmet à Tempo

- Tempo : stocke les traces distribuées, les rend requêtables via TraceQL dans Grafana
- Wazuh Manager + Elasticsearch : indexe les alertes de sécurité dans ES pour requête via datasource Elasticsearch dans Grafana

c. Couche 3 — Grafana (hub central)

Grafana est configuré avec 5 datasources : Prometheus, Loki, Tempo, Elasticsearch et AlertManager. Il héberge l'ensemble des dashboards, le moteur d'alerting unifié (Grafana Unified Alerting), les runbooks et la playlist NOC. C'est l'unique interface de supervision.

d. Couche 4 — Consommateurs

- Équipe SRE : dashboards USE et RED, alertes P1–P4
- Équipe NOC opérationnel : playlist kiosk mode, runbooks
- Équipe Sécurité : dashboard SOC Wazuh
- Direction : dashboard exécutif SLO, Error Budget, KPIs

2.3. Flux de données

Source	Protocole	Collecteur	Destination Grafana
Node Exporter / cAdvisor	HTTP pull (scrape)	Prometheus	Datasource Prometheus
Django / Nginx	HTTP pull + push OTLP	Prometheus + OTel Collector	Prometheus + Tempo
PostgreSQL	HTTP pull	Prometheus	Datasource Prometheus
Fichiers de logs	Push (Promtail)	Loki	Datasource Loki
Traces applicatives	Push OTLP/gRPC	OTel Collector → Tempo	Datasource Tempo
Événements sécurité	Push → Elasticsearch	Wazuh Manager	Datasource Elasticsearch

Source	Protocole	Collecteur	Destination Grafana
Alertes actives	HTTP pull	AlertManager	Datasource AlertManager

Point de vigilance : Wazuh nécessite Elasticsearch 8.x. Ne pas utiliser OpenSearch ni ES 7.x — incompatibilité certaine avec l'API Wazuh 4.7.

2.4. Guide défini des trois machines principales

Le guide définit trois machines principales et leurs rôles. La machine `nocserv` concentre les services de supervision, d'agrégation et de présentation, tandis que les machines `appserv` et `dbserv` exposent les métriques et les journaux de leurs propres systèmes.

Machine	Rôle principal	Adresse IP
nocserv	Prometheus, Alertmanager, Grafana, Loki, Tempo, Elasticsearch, Wazuh, cAdvisor	192.168.6.10/24
appserv	Application, Node Exporter, Promtail, cAdvisor	192.168.6.20/24
dbserv	PostgreSQL, Node Exporter, pg_exporter, Promtail	192.168.6.30

2.5. Logique d'installation

Le guide adopte une stratégie hybride. Les composants les plus courants de supervision sont installés en natif pour garder un contrôle précis sur les services `systemd`, les fichiers de configuration et les ports d'écoute. Les briques plus lourdes ou plus facilement isolables sont déployées avec Docker afin de simplifier l'isolation, les mises à jour et la maintenance.

Outil	Méthode	Machines	Ports
Prometheus	Natif	nocserv	9090
Alertmanager	Natif	nocserv	9093
Grafana	Natif	nocserv	3030
Node Exporter	Natif	appserv, dbserv	9100
Promtail	Natif	appserv, dbserv	9080
Pg_exporter	Natif	dbserv	9187
Loki	Docker	nocserv	3100
Tempo (OLTP gRPC)	Docker	nocserv	4317
Elasticsearch + Wazuh	Docker	nocserv	9200 (elasticsearch)
cAdvisor	Docker	nocserv, appserv	8080

3. Préparation de l'environnement (Debian + Docker)

3.1. Prérequis communs

Les trois machines reçoivent d'abord les mises à jour système, les outils de base et la configuration d'identité réseau. Cette étape garantit une base homogène avant le déploiement des services.

La normalisation des hostname et du fichier « */etc/hosts* » simplifie la résolution locale des noms ***nocserv***, ***appserv*** et ***dbserv***. Elle évite de multiplier les adresses IP dans les scripts et rend les configurations plus lisibles.

```
# Mise à jour système
sudo apt update && sudo apt upgrade -y

# Outils de base
sudo apt install -y curl wget tar gzip unzip gnupg2 ca-certificates \
    lsb-release software-properties-common apt-transport-https \
    net-tools ufw systemd

# Configurer les hostnames (adapter selon la machine)
# Sur noc :
sudo hostnamectl set-hostname nocserv
# Sur app :
sudo hostnamectl set-hostname appserv
# Sur db :
sudo hostnamectl set-hostname dbserv

# Ajouter les entrées dans /etc/hosts (sur chaque machine)
sudo tee -a /etc/hosts <<'EOF'
192.168.6.10  nocserv
192.168.6.20  appserv
192.168.6.30  dbserv
EOF
```

Figure 1 - Prérequis communs (toutes les machines)

3.2. Installation Docker (machines NOC & APP)

```
# Sur noc (192.168.6.10)

# Installer Docker
sudo apt remove -y docker docker-engine docker.io containerd runc 2>/dev/null || true

curl -fsSL https://download.docker.com/linux/debian/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/debian $(lsb_release -cs) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Ajouter l'utilisateur courant au groupe docker
sudo usermod -aG docker $USER
newgrp docker

# Activer Docker au démarrage
sudo systemctl enable --now docker
```

Figure 2 - Installation docker (machines NOC & APP)

Loki, Tempo, Elasticsearch, Wazuh et cAdvisor tournent en Docker sur **nocserv** tandis que cAdvisor tourne sur **appserv**.

Pour la vérification docker :

```
docker --version
docker compose version
sudo systemctl status docker
docker run --rm hello-world
```

Figure 3 - Vérification Docker

4. Description détaillée des services et outils

4.1. Prometheus

Prometheus est le cœur de la collecte de métriques. Il interroge périodiquement les cibles déclarées dans `prometheus.yml` via le mécanisme de scraping. Dans ce guide, il collecte ses propres métriques, celles de Node Exporter sur `noc/app/db`, celles de `pg_exporter` sur `db`, celles de `cAdvisor` sur `app`, ainsi que les indicateurs de l'application Django et d'Alertmanager.

Son fonctionnement repose sur des jobs de collecte, des labels pour identifier les hôtes et les rôles, et des règles d'alerte évaluées toutes les 15 secondes. Les données sont stockées dans une base TSDB locale avec une rétention de 30 jours.

4.2. Node Exporter

Node Exporter s'exécute sur les machines `app` et `db` pour exposer les métriques système de bas niveau : CPU, mémoire, disque, réseau, charge, système init, processus et autres statistiques du noyau.

Il ne stocke rien lui-même. Il publie simplement un endpoint HTTP `/metrics` sur le port 9100. Prometheus le scrute ensuite pour transformer ces valeurs brutes en séries temporelles.

4.3. Promtail

Promtail agit comme collecteur et expéditeur de journaux. Il lit les fichiers de log système (`/var/log/syslog`, `/var/log/auth.log`, `/var/log/kern.log`) et le journal `systemd`, puis il envoie les événements vers Loki.

Promtail enrichit les logs avec des labels comme `host`, `job` et `env`. Ce marquage est essentiel car il permet ensuite à Grafana et à Loki de filtrer les logs par machine, par type de journal ou par environnement.

4.4. Loki

Loki est le moteur central de stockage et de recherche des logs. Contrairement à un moteur d'indexation complet comme Elasticsearch, Loki s'appuie fortement sur les labels pour indexer les flux de logs tout en gardant une empreinte plus légère.

Dans le guide, Loki reçoit les journaux envoyés par Promtail via l'API `/loki/api/v1/push`. Il expose ensuite des requêtes, notamment pour Grafana, afin de corréler les logs avec les métriques et les traces.

4.5. Tempo

Tempo est la solution de traçage distribué. Elle reçoit les traces applicatives au format OTLP, ainsi que d'autres formats comme Jaeger et Zipkin. Son rôle est de reconstituer le parcours d'une requête à travers les services.

Tempo sert ici à analyser les latences, à identifier les goulots d'étranglement et à relier une requête lente à ses logs et à ses métriques. Le guide active aussi des générateurs de métriques de service et de span-metrics pour améliorer l'observabilité.

4.6. Elasticsearch et Wazuh

Elasticsearch fournit le moteur de stockage des événements de sécurité. Wazuh s'appuie sur ce stockage pour centraliser les alertes, les événements d'agents et les informations de détection.

Wazuh complète la supervision classique en ajoutant une dimension sécurité : analyse d'événements, corrélation, détection d'anomalies et tableau de bord dédié. Le premier démarrage peut être plus long, car les index et les services internes se préparent progressivement.

4.7. cAdvisor

cAdvisor cAdvisor observe les conteneurs Docker et collecte des métriques sur la consommation CPU, mémoire, réseau, stockage et état des conteneurs. Il joue un rôle clé pour superviser la charge des services conteneurisés sur noc et app.

Il expose ses métriques en HTTP sur le port 8080. Prometheus peut ensuite les scruter pour afficher des tableaux de bord sur les conteneurs, les images et la consommation des ressources.

4.8. Grafana

Grafana est la couche de visualisation et de corrélation. C'est le point d'entrée pour consulter les tableaux de bord, construire des panneaux, explorer les métriques et naviguer entre métriques, logs et traces.

Dans le guide, Grafana reçoit comme sources de données Prometheus, Loki et Tempo. C'est ce lien qui donne de la cohérence à l'ensemble : les métriques décrivent l'état, les logs expliquent les événements, et les traces montrent le chemin d'exécution.

4.9. Alertmanager

Alertmanager reçoit les alertes générées par Prometheus. Son travail n'est pas de détecter les problèmes, mais de les organiser, les regrouper, les dédupliquer et les transmettre aux bons destinataires.

La configuration du guide montre des routes différentes selon la sévérité, avec un traitement particulier pour les alertes critiques. C'est lui qui transforme une simple règle Prometheus en notification exploitable.

5. Guide d'installation et de configuration des outils + vérification

5.1. Procédure d'installation des services

Les étapes détaillées d'installation, de configuration et de déploiement des différents composants et services présentés précédemment sont disponibles dans le fichier Markdown joint à ce document (« ***noc-install-guide.md*** »).

Ce fichier contient l'ensemble des commandes, configurations et procédures nécessaires à la mise en place de la plateforme de supervision. Il constitue le guide technique d'installation associé à ce rapport.

La partie suivante présente les différentes vérifications réalisées après l'installation afin de valider le bon fonctionnement des services déployés.

5.2. Vérification globale du stack

a. Services natifs (nocserv)

```
noc@nocserv:/opt/noc-stack$ echo "=== Services natifs ==="
for svc in prometheus alertmanager grafana-server; do
  STATUS=$(systemctl is-active $svc)
  echo "$svc : $STATUS"
done
=== Services natifs ===
prometheus : active
alertmanager : active
grafana-server : active
noc@nocserv:/opt/noc-stack$
```

Figure 4 - Vérification des services natifs sur nocserv

b. Services natifs (appserv / dbserv)

Vérification faite sur appserv et dbserv :

```

app@appserv:~$ echo "=== Agents ==="
for svc in node_exporter promtail; do
    STATUS=$(systemctl is-active $svc)
    echo "$svc : $STATUS"
done
=== Agents ===
node_exporter : active
promtail : active
app@appserv:~$

```

Figure 5 - Vérification des services natifs sur appserv

```

db@dbserv:~$ echo "=== Agents ==="
for svc in node_exporter promtail; do
    STATUS=$(systemctl is-active $svc)
    echo "$svc : $STATUS"
done
=== Agents ===
node_exporter : active
promtail : active
db@dbserv:~$

```

Figure 6 - Vérification des services natifs sur dbserv

c. Conteneurs docker (nocserv)

```

noc@nocserv:/opt/noc-stack$ echo "=== Conteneurs Docker ==="
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
=== Conteneurs Docker ===
NAMES                STATUS                PORTS
wazuh-manager        Up 16 hours           0.0.0.0:514->514/udp, [::]:514->514/udp
cp, 0.0.0.0:1515->1515/tcp, [::]:1515->1515/tcp, 0.0.0.0:1514->1514/udp, [::]:151
udp, 1516/tcp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp
wazuh-dashboard      Up 33 seconds         443/tcp, 0.0.0.0:5601->5601/tcp, [::]:5
1/tcp
elasticsearch         Up 21 hours (healthy) 0.0.0.0:9200->9200/tcp, [::]:9200->9200
0.0.0.0:9300->9300/tcp, [::]:9300->9300/tcp
loki                  Up 21 hours           0.0.0.0:3100->3100/tcp, [::]:3100->3100
0.0.0.0:9095->9095/tcp, [::]:9095->9095/tcp
tempo                 Up 22 hours           0.0.0.0:3200->3200/tcp, [::]:3200->3200
0.0.0.0:4317-4318->4317-4318/tcp, [::]:4317-4318->4317-4318/tcp, 0.0.0.0:9411->9411
]:9411->9411/tcp, 0.0.0.0:14250->14250/tcp, [::]:14250->14250/tcp
cadvisor              Up 26 hours (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080
noc@nocserv:/opt/noc-stack$

```

Figure 7 - Vérification des conteneurs docker sur nocserv

d. Endpoints de santé

```

noc@nocserv:/opt/noc-stack$ echo "=== Santé des services ==="
curl -s http://localhost:9090/-/healthy && echo " Prometheus OK"
curl -s http://localhost:9093/-/healthy && echo " Alertmanager OK"
curl -s http://localhost:3030/api/health && echo ""
curl -s http://localhost:3100/ready && echo " Loki OK"
curl -s http://localhost:3200/ready && echo " Tempo OK"
curl -s http://localhost:8080/healthz && echo " cAdvisor OK"
curl -s http://localhost:9200/_cluster/health | python3 -c "import sys,json; d=json.load(sys.stdin); print(f' Elasticsearch : {d[\"status\"]}')"
=== Santé des services ===
Prometheus Server is Healthy.
Prometheus OK
OK Alertmanager OK
{
  "database": "ok",
  "version": "13.0.2",
  "commit": "3fcd5a"
}
ready
Loki OK
ready
Tempo OK
ok cAdvisor OK
Elasticsearch : green

```

Figure 8 - Vérification des endpoints de santé

e. Targets prometheus

```

noc@nocserv:/opt/noc-stack$
echo "=== Targets Prometheus ==="
curl -s "http://localhost:9090/api/v1/targets" \
  | python3 -c "
import sys, json
data = json.load(sys.stdin)
for t in data['data']['activeTargets']:
    print(f\" {t['labels']['job']} | {t['scrapeUrl']} | {t['health']}\")
"
=== Targets Prometheus ===
alertmanager | http://localhost:9093/metrics | up
cadvisor | http://192.168.6.20:8080/metrics | up
django-app | http://192.168.6.20:8000/metrics | up
node_exporter | http://192.168.6.10:9100/metrics | up
node_exporter | http://192.168.6.20:9100/metrics | up
node_exporter | http://192.168.6.30:9100/metrics | up
pg_exporter | http://192.168.6.30:9187/metrics | up
prometheus | http://localhost:9090/metrics | up

```

Figure 9 - Vérification des targets Prometheus

6. L3 — Les 7 dashboards Grafana

6.1. Dashboard USE — Infrastructure

Le dashboard USE (Utilization, Saturation, Errors) surveille les ressources système selon la méthode USE de Brendan Gregg. Il est alimenté par Node Exporter et cAdvisor.

Panneaux à créer

Panneau	Type	Query PromQL
CPU Utilization (%)	Gauge	100 - (avg by(host)(rate(node_cpu_seconds_total{mode="idle", host=~"\$host"}[\$__interval])) * 100)
CPU Saturation (run queue)	Time series	node_load1{host=~"\$host"}
Memory Utilization (%)	Stat	100 * (1 - (node_memory_MemAvailable_bytes{host=~"\$host"} / node_memory_MemTotal_bytes{host=~"\$host"}))
Disk I/O Saturation	Time series	rate(node_disk_io_time_seconds_total[\$__interval])
Network In	Bar gauge	rate(node_network_receive_bytes_total{device!="lo", host=~"\$host"}[\$__interval]) / 1024
Network Out	Bar gauge	rate(node_network_transmit_bytes_total{device!="lo", host=~"\$host"}[\$__interval]) / 1024
Network Errors/s	Time series	rate(node_network_receive_errs_total{device!="lo", host=~"\$host"}[\$__interval])

Panneau	Type	Query PromQL
Container CPU Usage	Table	sum by(name)(rate(container_cpu_usage_seconds_total[5m])) * 100
Container Memory	Bar gauge	sum by(name)(container_memory_working_set_bytes)
Disk Usage (%)	Gauge + thresholds	(1 - (node_filesystem_avail_bytes{fstype!="tmpfs", mountpoint="/", host=~"\$host"} / node_filesystem_size_bytes{fstype!="tmpfs", mountpoint="/", host=~"\$host"})) * 100

Seuils (thresholds) recommandés

- CPU Utilization : vert < 70%, orange 70-85%, rouge > 85%
- Memory : vert < 75%, orange 75-90%, rouge > 90%
- Disk : vert > 30%, orange 15-30%, rouge < 15%



Figure 10 - Dashboard USE interface dans Grafana

6.2. Dashboard RED — Application Django

Le dashboard RED (Rate, Errors, Duration) mesure la santé de l'application Django selon la méthode RED de Tom Wilkie, en combinant métriques Prometheus, logs Loki et traces Tempo.

Panneaux à créer

Panneau	Type	Source
Request Rate (req/s)	Time series	<code>sum(rate(django_http_responses_total[5m]))</code>
Error Rate 5xx (%)	Stat + alerte	<code>sum(rate(django_http_responses_total{status=~'5..'}[5m])) / sum(rate(django_http_responses_total[5m])) * 100</code>
P50 / P95 / P99 Latency	Time series	<code>histogram_quantile(0.99, rate(django_http_requests_latency_seconds_bucket[5m]))</code>
Request par endpoint	Bar chart	<code>topk(10, sum by(view)(rate(django_http_responses_total[5m])))</code>
Log errors (Loki)	Logs panel	<code>{job='django'} = 'ERROR'</code>
Traces récentes (Tempo)	Table	<code>TraceQL : {http.status_code >= 500}</code>
Active DB connections	Gauge	<code>django_db_new_connections_total</code>
Uptime / Status	Stat	<code>up{job='django-app'}</code>

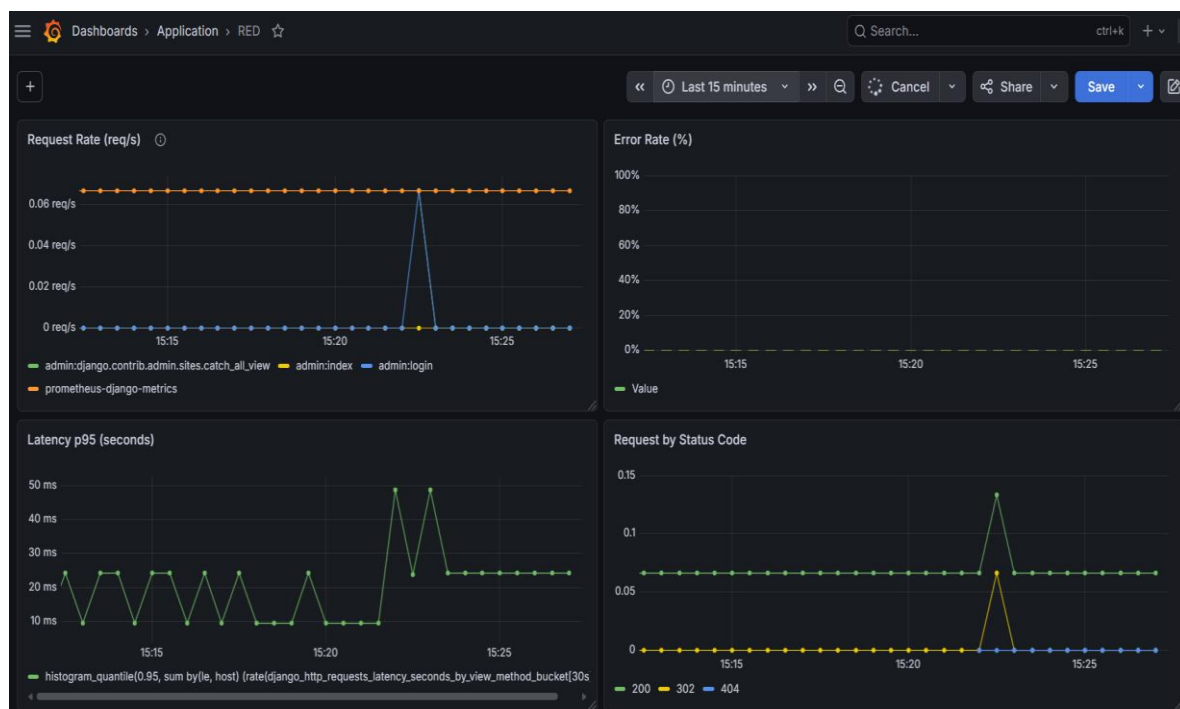


Figure 11 - Dashboard RED : Application Django

6.3. Dashboard PostgreSQL

Panneau	Query
Connexions actives	pg_stat_activity_count
Transactions/s (Commit)	rate(pg_stat_database_xact_commit[\$__interval])
Rollback/s	rate(pg_stat_database_xact_rollback[\$__interval])
Locks actifs	pg_locks_count
Cache hit ratio	pg_stat_database_blks_hit/(pg_stat_database_blks_hit+pg_stat_database_blks_read)
DB Size	sum(pg_database_size_bytes)
Dead tuples (utile pour les grosse base)	pg_stat_user_tables_n_dead_tup

Panneau	Query
WAL size	pg_wal_size_bytes
Slow queries (Loki)	{job='postgres'} = 'duration' duration > 1s

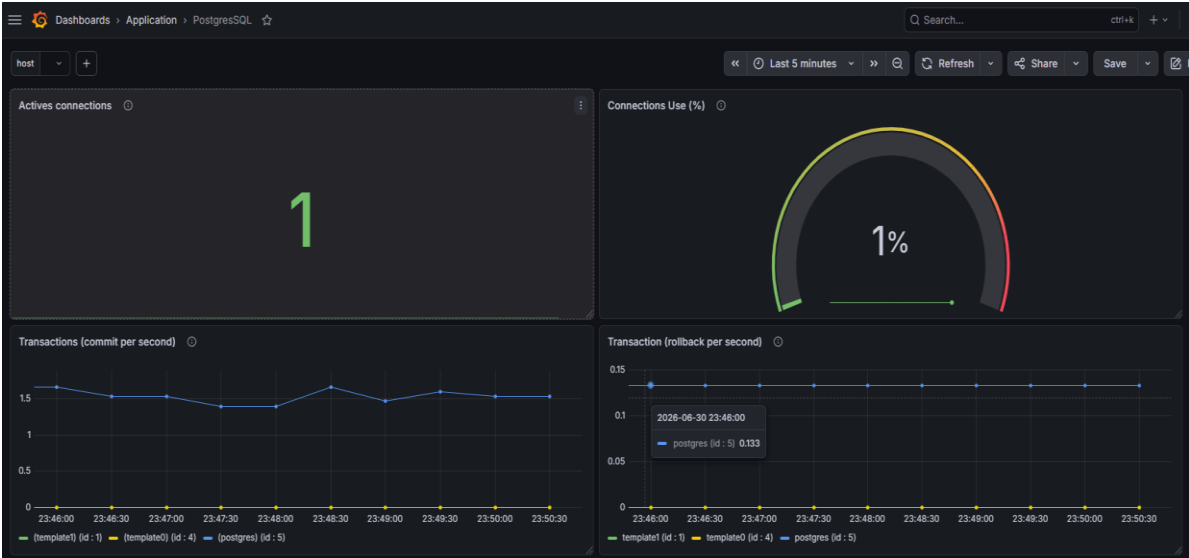


Figure 12 - Dashboard PostgreSQL

6.4. Dashboard SOC — Wazuh

Le dashboard SOC interroge Elasticsearch via la datasource dédiée pour visualiser les événements de sécurité collectés par Wazuh.

Panneau	Type	Query ES
Alertes par niveau (1-15)	Bar chart	terms: rule.level
Top 10 règles déclenchées	Table	terms: rule.description, order: count desc
Événements FIM (File Integrity)	Logs	filter: rule.groups: syscheck
Alertes géo (map)	Geomap	terms: GeoLocation.country_name

Panneau	Type	Query ES
Score de conformité PCI-DSS	Gauge	filter: rule.pci_dss: *
Connexions SSH échouées	Time series	filter: rule.groups: sshd AND rule.level >= 5
Timeline alertes critiques	Time series	filter: rule.level >= 12, interval: 1m
Processus suspects	Table	filter: rule.groups: process_monitoring

6.5. Dashboard SLO + Error Budget

Ce dashboard implémente le concept SRE de SLO (Service Level Objective) et Error Budget. Le statut RAG (Rouge/Amber/Vert) donne une vue immédiate de l'état de santé des services.

```
# SLO Availability (30 jours glissants)
# Objectif : 99.5% de requêtes sans erreur 5xx

# Calcul SLO actuel
1 -
(sum(increase(django_http_responses_total{status=~'5..'}[
30d]))
  / sum(increase(django_http_responses_total[30d])))

# Error Budget restant (%)
# Budget total = 1 - 0.995 = 0.005 (0.5% d'erreurs
autorisées)
# Error Budget restant =
((0.005 -
(sum(increase(django_http_responses_total{status=~'5..'}[
30d]))
```

```
sum(increase(django_http_responses_total[30d])) / 0.005) * 100
```

Panneaux SLO

- SLO Availability : stat panel avec seuils vert > 99.5%, amber 99-99.5%, rouge < 99%
- Error Budget restant : gauge 0-100%, rouge < 0% (budget épuisé)
- Burn rate 1h/6h/24h : time series — détecte une consommation accélérée du budget
- SLO Latence P99 < 500ms : stat panel
- Historique 90 jours : time series mensuel

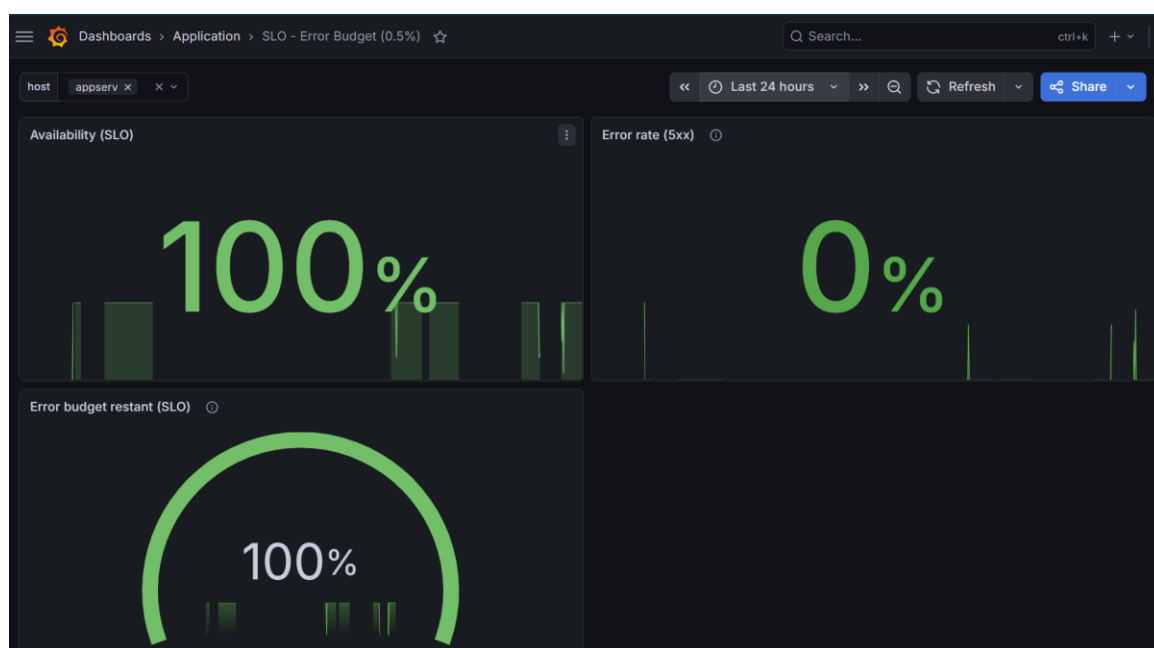


Figure 13 - Dashboard SLO Error budget

6.6. Dashboard Exécutif

Vue synthétique destinée à la direction. Toutes les métriques clés en un coup d'œil, sans détail technique.

- Disponibilité globale 30j (% avec statut RAG)

- Nombre d'incidents P1 ce mois
- MTTR moyen (Mean Time To Recover)
- Error Budget consommé (%)
- Coût infrastructure estimé (si données disponibles)
- Satisfaction utilisateur (si NPS disponible)

7. Bonus — Déploiement Ansible (IaC)

Afin d'automatiser et de simplifier le déploiement de la plateforme de supervision, une approche basée sur Ansible a été mise en place.

Ansible permet de transformer l'installation complète du NOC en une procédure automatisée et reproductible. Grâce à cette approche Infrastructure as Code (IaC), il devient possible de déployer l'ensemble des composants nécessaires (services, configurations et dépendances) sur une nouvelle machine virtuelle Debian à l'aide d'une seule commande.

Cette automatisation permet de garantir une installation homogène, de réduire les erreurs de configuration manuelle et de faciliter la maintenance ainsi que la reproduction de l'environnement sur d'autres infrastructures.

L'ensemble du projet Ansible, incluant les playbooks, les rôles et les fichiers de configuration associés, est disponible dans le [dépôt GitHub du projet](#).

Conclusion

Cette installation met en place une plateforme NOC moderne, structurée et extensible. Prometheus, Alertmanager, Grafana, Loki, Tempo, Node Exporter, Promtail, cAdvisor, Elasticsearch et Wazuh forment un ensemble complémentaire qui permet de surveiller l'infrastructure, d'analyser les incidents et d'améliorer la réactivité opérationnelle.

Table des matières

Tables des figures.....	2
Sommaire	3
1. Présentation du projet	4
1.1. Contexte pédagogique.....	4
1.2. Mise en situation.....	4
1.3. Stack technique	4
1.4. Livrables attendus	5
1.5. Environnement de travail.....	5
2. Architecture globale du NOC.....	7
2.1. Principe architectural	7
2.2. Les quatre couches	7
a. Couche 1 — Sources	7
b. Couche 2 — Collecteurs.....	7
c. Couche 3 — Grafana (hub central).....	8
d. Couche 4 — Consommateurs	8
2.3. Flux de données	8
2.4. Guide défini des trois machines principales.....	9
2.5. Logique d'installation	9
3. Préparation de l'environnement (Debian + Docker).....	11
3.1. Prérequis communs.....	11
3.2. Installation Docker (machines NOC & APP).....	12
4. Description détaillée des services et outils	13
4.1. Prometheus	13
4.2. Node Exporter	13
4.3. Promtail	13

4.4.	Loki	13
4.5.	Tempo	14
4.6.	Elasticsearch et Wazuh	14
4.7.	cAdvisor	14
4.8.	Grafana	15
4.9.	Alertmanager	15
5.	Guide d'installation et de configuration des outils + vérification	16
5.1.	Procédure d'installation des services	16
5.2.	Vérification globale du stack	16
a.	Services natifs (nocserv)	16
b.	Services natifs (appserv / dbserv)	16
c.	Conteneurs docker (nocserv)	17
d.	Endpoints de santé	17
e.	Targets prometheus	18
6.	L3 — Les 7 dashboards Grafana	19
6.1.	Dashboard USE — Infrastructure	19
6.2.	Dashboard RED — Application Django	21
6.3.	Dashboard PostgreSQL	22
6.4.	Dashboard SOC — Wazuh	23
6.5.	Dashboard SLO + Error Budget	24
6.6.	Dashboard Exécutif	25
7.	Bonus — Déploiement Ansible (IaC)	27
	Conclusion	28
	Table des matières	29